PROYECTO INTERMODULAR II — 2.º DAW
Tarea 2: Propuesta y Planificación
Documento 04: Arquitectura Objetivo y Tratamiento de Datos
Alumno: Damián Conejos
Caso: Panel de Gestión de Entradas y Aforos de TechFest

1. Arquitectura objetivo (5 Elementos)

La solución modernizada sustituye las plantillas Blade y los scripts de jQuery heredados de Attendize v2.8.0 por una arquitectura por capas desacoplada pero cohesionada mediante Inertia.js:

Responsabilidades de cada elemento:

1. **Navegador Web:**
    - Interpreta y ejecuta el código JavaScript compilado y gestiona los eventos de usuario (clics, pulsaciones de teclado, scroll).
    - Almacena las cookies de sesión y mantiene en memoria el árbol DOM de la página sin forzar recargas completas.
2. **React con TypeScript:**
    - Renderiza los componentes visuales de la interfaz (tarjetas de tickets, modales, alertas y botones) de forma reactiva según su estado local.
    - Valida en tiempo real los datos introducidos en el formulario y asegura la consistencia de tipos en el frontend gracias a las interfaces de TypeScript.
3. **Inertia.js:**
    - Actúa como puente de comunicación entre el frontend de React y el backend de Laravel sin necesidad de construir una API REST separada.
    - Intercepta las llamadas del usuario y envía peticiones HTTP asíncronas por debajo, actualizando únicamente las propiedades (*props*) que han cambiado en la vista.
4. **Laravel:**
    - Aplica la autenticación de usuarios, la autorización de acceso al evento TechFest y las reglas de validación en el servidor mediante *Form Requests*.

- Procesa la lógica de negocio (comprobar límites de aforo, activar pausas) y gestiona las transacciones con la base de datos a través de Eloquent.

5. **MySQL:**
    - Garantiza la persistencia relacional, la integridad referencial y el almacenamiento atómico de la información del evento y los tickets.
    - Ejecuta las consultas de lectura, inserción y actualización solicitadas por el servidor.

2. Recorrido conceptual de una petición (Ejemplo: Guardar nuevo ticket)

1. **Acción en el navegador:** El usuario rellena los datos en el modal de crear ticket y pulsa el botón «Guardar».
2. **Validación en React:** El componente comprueba en local que el título no esté vacío y que el aforo sea un entero positivo. Si es válido, se invoca el método de envío.
3. **Tránsito con Inertia.js:** Inertia empaqueta los datos del formulario y realiza una petición HTTP POST hacia el servidor enviando la cabecera especial X-Inertia, impidiendo que el navegador haga un refresco blanco de página.
4. **Procesamiento en Laravel:** Laravel intercepta la petición, verifica que el usuario es el dueño de TechFest, valida de nuevo los campos en el backend y construye el modelo correspondiente.
5. **Persistencia en MySQL:** Laravel ejecuta una sentencia INSERT INTO tickets ... en la base de datos MySQL, asignando un nuevo identificador único.
6. **Retorno y renderizado:** Laravel responde a Inertia con los datos actualizados del evento. Inertia inyecta el nuevo ticket en las propiedades de React y la interfaz muestra de inmediato la nueva tarjeta junto con un mensaje Toast de éxito.

3. Tratamiento de los datos

Entidades y relaciones implicadas

El recorrido modernizado interactúa principalmente con dos entidades centrales heredadas de Attendize:

- **events (Eventos):** Representa el evento TechFest (id, title, start_date, end_date, user_id). Relación 1:N con tickets.
- **tickets (Tipos de entrada / talleres):** Contiene cada pase o cupo (id, event_id, title, price, quantity_available, quantity_sold, is_paused). Pertenece a un evento (belongsTo).
- *(Relaciones de integridad indirectas)*: Las tablas orders y attendees se consultan para comprobar si un ticket tiene ventas antes de permitir su edición de aforo a la baja o su borrado.

Qué datos se conservan, cambian o se añaden

- **Se conservan tal cual:** La estructura base de las tablas events y tickets de Attendize v2.8.0 para asegurar la compatibilidad con el resto del proyecto Docker desplegado en la Tarea 1.
- **Se transforman / adaptan:**
    - El campo price: Attendize lo almacena a menudo como decimal; en el frontend se formatea de forma homogénea con el símbolo del euro (€) y dos decimales fijos en React.
    - El campo is_paused: En Attendize v2.8.0 a veces se manejaba mediante borrado lógico o fechas de venta expiradas. En la modernización se consolida como un campo booleano limpio (0 = En venta, 1 = Pausado).
- **Validaciones estrictas que se aplican:**
    - title: Obligatorio, cadena de texto, mínimo 3 caracteres, máximo 120 caracteres.
    - quantity_available: Obligatorio, número entero, valor mínimo 1. Al editar: quantity_available >= quantity_sold.
    - price: Obligatorio, valor numérico decimal mayor o igual a 0.00.
    - max_per_person: Entero entre 1 y 10.

Tabla de correspondencia entre modelo antiguo y nuevo:

Campo en Attendize v2.8.0	Campo en la Solución Modernizada	Tipo de dato MySQL	Función / Regla de negocio
tickets.id	ticket.id	INT UNSIGNED	Clave primaria autoincremental.
tickets.event_id	ticket.eventId	INT UNSIGNED	Clave foránea referenciando a events.id.
tickets.title	ticket.title	VARCHAR(255)	Denominación del ticket o taller de TechFest.
tickets.price	ticket.price	DECIMAL(8, 2)	Importe de la entrada (0.00 para gratuitas).
tickets.quantity_available	ticket.quantityAvailable	INT	Cupo total de plazas asignadas a ese tipo.
tickets.quantity_sold	ticket.quantitySold	INT	Entradas ya reservadas (solo lectura en este módulo).
tickets.is_paused	ticket.isPaused	TINYINT(1) / BOOL	0: Venta activa pública; 1: Venta pausada.

Reglas para valores incompletos o duplicados

1. **Valores incompletos (nulos o vacíos):**
    - Si el usuario no introduce precio, el sistema le asigna por defecto 0.00.
    - Si no se especifica el límite por persona (max_per_person), se fuerza por defecto a 1.
    - Si el nombre del ticket se envía en blanco o con solo espacios, el formulario rechaza el envío marcando el campo como obligatorio.
2. **Duplicidad de títulos:**
    - No se permite crear dos tickets con el mismo nombre exacto para la misma edición de TechFest (por ejemplo, dos entradas llamadas exactamente *"Pase General"*). El backend comprueba si ya existe un registro con idéntico title y event_id devolviendo un aviso antes de insertar.
3. **Protección de registros con ventas:**
    - Si un ticket ya tiene quantity_sold > 0, se bloquea su eliminación física para no generar datos huérfanos en las tablas de pedidos y asistentes.